

MEDIATEK

MT8676 Android DVR User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Dandan Hu	Official release

Table of Contents

Version History 2

Table of Contents 3

List of Figures 3

1 DVR 4

 1.1 Overview 4

 1.1.1 Brief Introduction 4

 1.1.2 DVR Abbreviations 4

 1.1.3 Main Features 4

 1.2 Architecture/Process Overview 5

 1.2.1 DVR Architecture 5

 1.2.2 DVR Flow 6

 1.2.3 MP4 Writer Flow 6

 1.3 Configuration/Customization Guideline 7

 1.3.1 Three Entry Points to the Recorder 7

 1.3.2 Common DVR Features 7

 1.4 Frequently Asked Questions/Troubleshooting 10

 1.4.1 Common DVR Issue Analysis 10

Exhibit 1 Terms and Conditions 14

List of Figures

Figure 1-1. DVR architecture 5

Figure 1-2. DVR flow 6

Figure 1-3. MP4 writer flow 7

Figure 1-4. Sub-stream flow 8

1 DVR

1.1 Overview

1.1.1 Brief Introduction

This section primarily introduces the architecture of the MT8676 DVR and the analysis of some common DVR issues.

Digital Video Recorder (DVR) is the built-in video recording feature of the SPM platform, supporting MP4 and ts recording as well as H264 data callback.

1.1.2 DVR Abbreviations

Abbreviation	Explanation
API	Application Programming Interface
DVR	Digital Video Recorder
SPM	Smart Platform

1.1.3 Main Features

1. Multi-channel Recording

A single camera can simultaneously record and generate multiple video files (including H264 callback).

2. File Segmentation

The recording file duration can be dynamically set, such as 1 minute, 3 minutes, 5 minutes, etc. Segmentation condition: while (Video duration >= max duration)

- If the audio duration is reached first, it will wait for the video duration.
- If the video duration is reached first, it will segment.

3. Main Stream

While writing to disk, callback a copy of H264 or ts data.

How to enable: Pass in the parameter VIDEO_FRAME_MODE_DUAL_SOURCE or VIDEO_FRAME_MODE_DUAL_PACKET when calling startRecord.

4. Sub-stream

Do not write to disk, and only callback a copy of H264 data or ts data.

How to enable: Pass in the parameter VIDEO_FRAME_MODE_SOURCE when calling startRecord.

5. File Protection

When a collision is triggered, a period of video will be saved automatically to a specified directory.

6. Keypoint Protection

Used to record the data from a certain period before and after the current time (e.g., 10 seconds before and after) to a specified directory.

7. Loop Recording

After DVR starts, it will continue recording indefinitely, automatically segmenting files at fixed intervals.

8. Loop Deletion

When the SD storage card is nearly full, it automatically deletes the oldest files.

9. File Header I Frame

The first frame of each segmented file is an I-frame to avoid screen distortion during playback.

10. Audio Mute

Audio recording can be dynamically turned on or off before and after recording.

1.2 Architecture/Process Overview

1.2.1 DVR Architecture

Figure 1-1 shows the DVR architecture. The Demo APK calls the SDK interface, which communicates with the server side through a binder call to the CarCamDeviceClient section. The CarCamDeviceClient interfaces with the CameraDeviceClient and RecorderMgr. The RecorderMgr interfaces with the SdcardCheck and StagefrightRecordSmp. The StagefrightRecordSmp interfaces with the MediaWriterSmp.

DVR Architecture

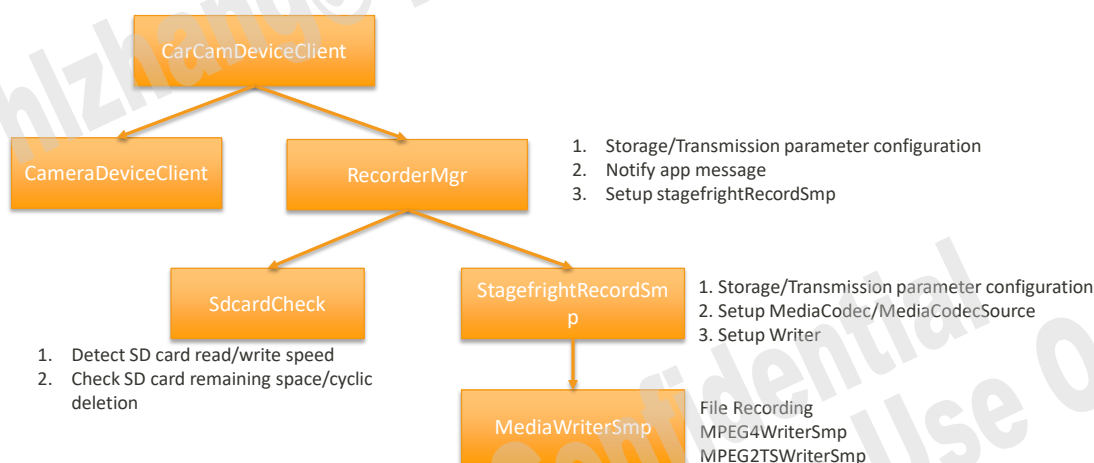


Figure 1-1. DVR architecture

1.2.2 DVR Flow

Figure 1-2 represents the overall flow of the DVR, taking MP4 recording as an example.

Firstly, recorderMgr is the recording manager, and a new recorderMgr instance is created for each recording channel. Under recorderMgr, a StagefrightRecorderSmp instance is instantiated. MediaCodecSource sends the data to the encoder for encoding, and once encoded, the data are returned through mediaCodecSource.

Video: The data source before encoding is the camera, which sends data to the bufferqueue. The data are then passed through graphicBufferSource and OMX to the encoder for encoding. The encoded data are in H264 format and then stored in MediaCodecSource.

Audio: The data source before encoding is the microphone. Data are retrieved from the mic and then encoded. The decoded data format is AAC.

Writer: There are two tracks, the video track and the audio track. The track continuously reads data from MediaCodecSource. According to the MP4 format, it packages the data, adds some specification information, and then writes it to the SD card file.

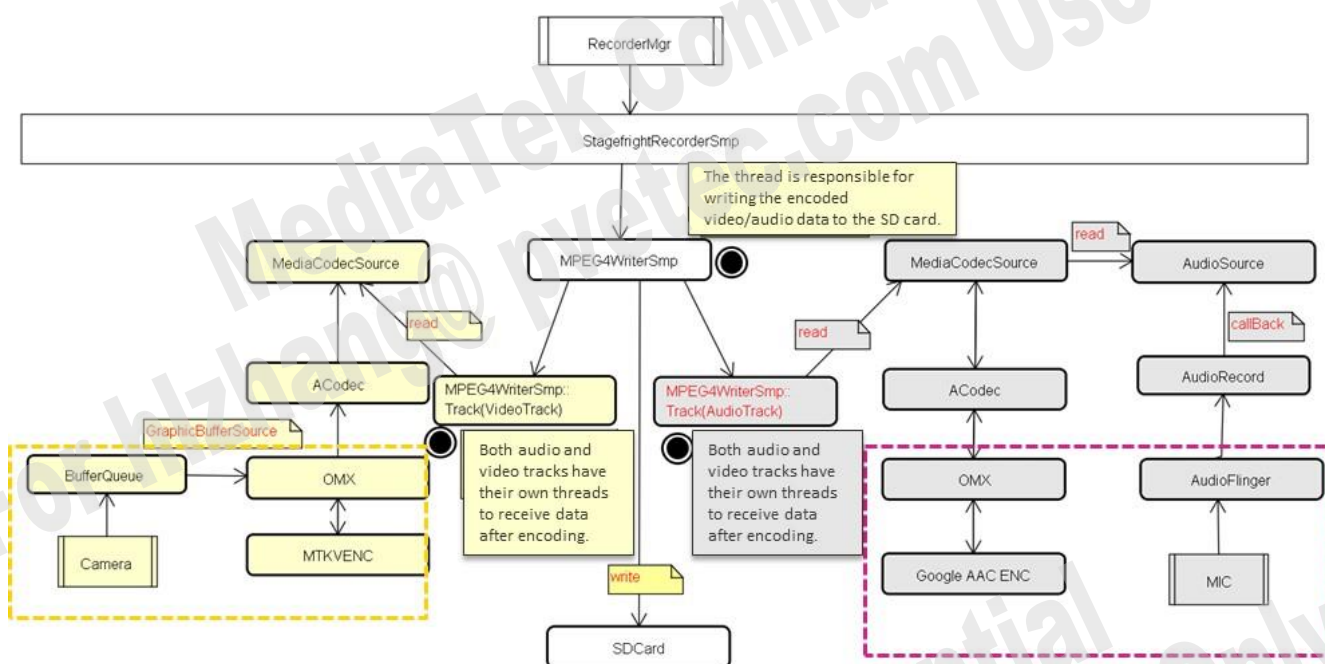


Figure 1-2. DVR flow

1.2.3 MP4 Writer Flow

Figure 1-3 represents the MP4 writer flow, which has no big difference from the TS flow. The difference is that due to the absence of header file information in TS, TS writer writes one frame of video followed by one frame of audio. TS writer selects the smaller PTS between video and audio and returns the index to determine whether to write video or audio next.

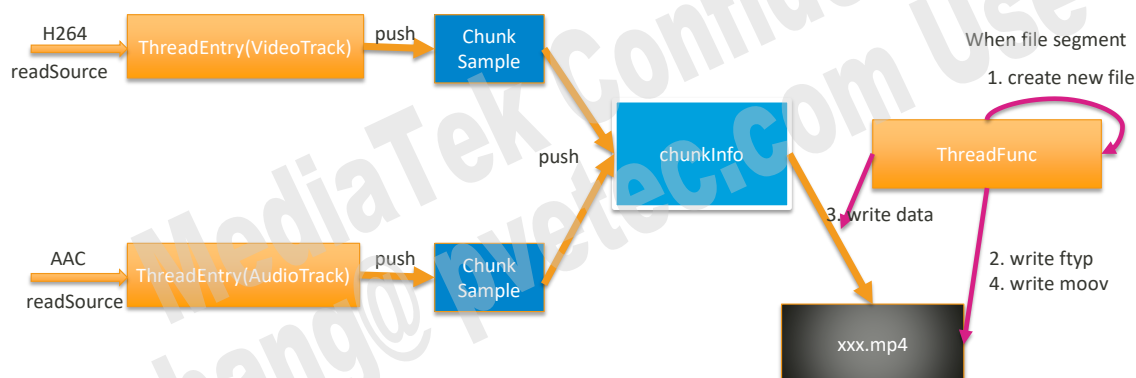


Figure 1-3. MP4 writer flow

1.3 Configuration/Customization Guideline

1.3.1 Three Entry Points to the Recorder

- 1) startRecord
- 2) stopRecord
- 3) notifyRecordEvent: Dynamically sets parameters

1.3.2 Common DVR Features

1.3.2.1 Sub-stream

The sub-stream is not written to disk; it only callbacks a copy of H264 data or TS data.

How to enable: Pass in the parameter VIDEO_FRAME_MODE_SOURCE when calling startRecord. The difference between normal recording and sub-stream is that normal recording writes to the disk, whereas the sub-stream callbacks the data. The recorderMgr has two outTracks that read data from mediaCodecSource, callback H264 and AAC data, and then notify the APP.

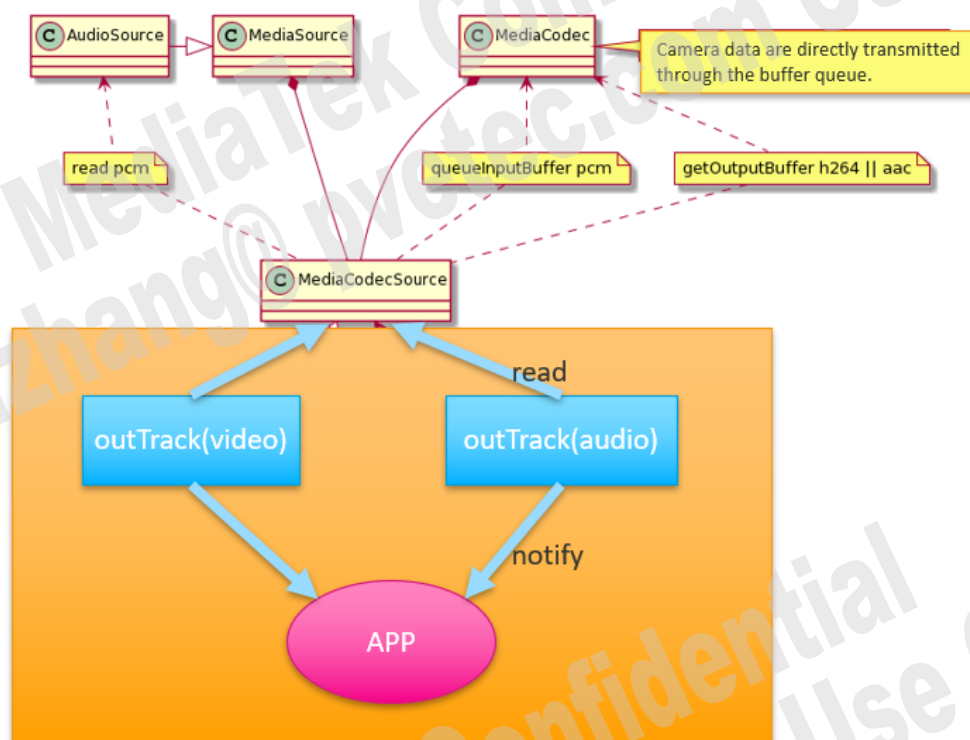


Figure 1-4. Sub-stream flow

1.3.2.2 Keypoint Protection

Feature: Protects a specified duration of video by moving it to the protect directory.

Application Scenarios:

Case 1. Manual marking: While driving, a user sees a particularly beautiful scene and wants to protect the video by manually triggering event marking protection.

Case 2. Passive marking: During driving, a sudden car collision occurs, and the user wants to protect the video from before and after the collision to determine accident responsibility. Gsensor collision triggers event marking protection.

Actual Operation: Extracts a specified duration of video from the currently recording video file, generates a new video file, and places it in the protect directory.

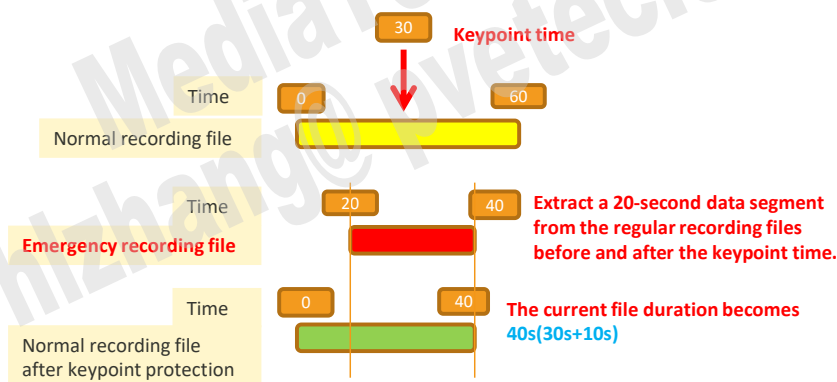
API:

lockRecordingVideo(int duration);

Parameter: The higher 16 bits of duration represent the forward protection duration, and the lower 16 bits represent the backward protection duration. Customers can specify the forward and backward protection durations; if no forward duration is set, the default forward protection duration is the same as the backward.

Keypoint protection has the following three cases:

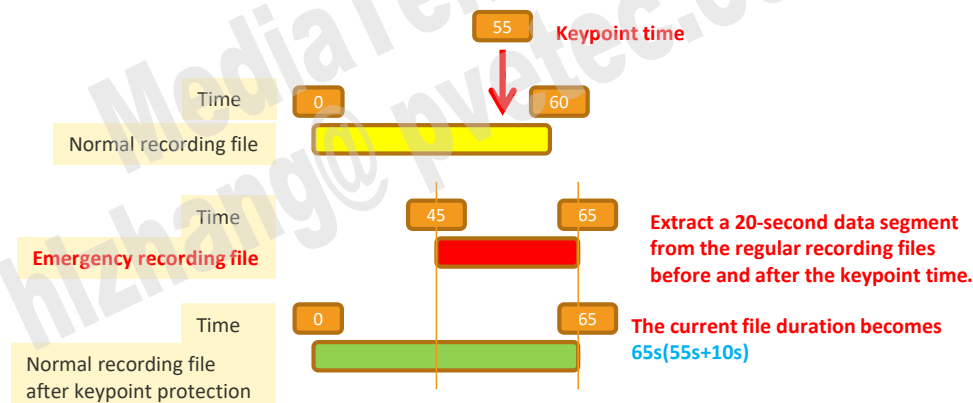
Keypoint Protection Case1



Use a 10-second pre and post protection as an example: The keypoint occurs at 30s, resulting in a regular recording duration that is less than the usual 60s. An emergency recording file of 20s will be created (10s before the keypoint + 10s after the keypoint).

- 1) The normal recording file after the keypoint will be saved to the sdcard/dcim/camera/front directory.
- 2) The emergency recording file will be moved to the sdcard/dcim/camera/protect/keypoint directory.

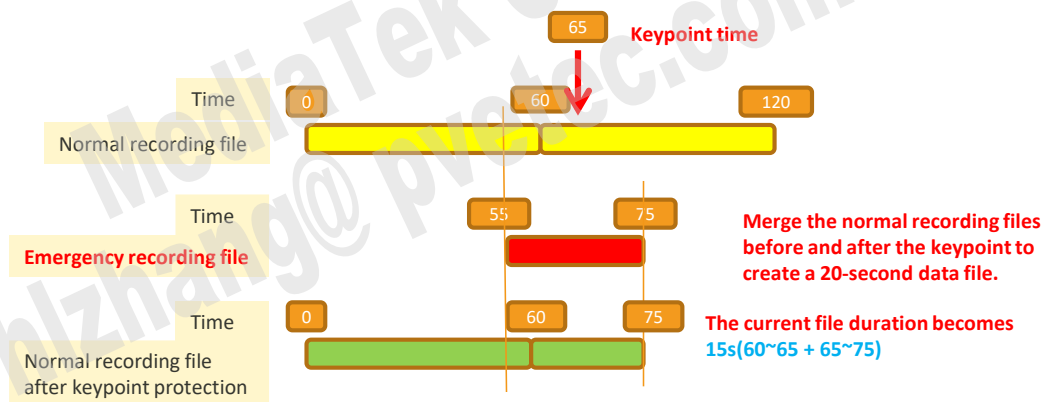
Keypoint Protection Case2



Use a 10-second pre and post protection as an example: The keypoint occurs at 55s, resulting in a regular recording duration that is less than the usual 60s. An emergency recording file of 20s will be created (10s before the keypoint + 10s after the keypoint).

- 1) The normal recording file after the keypoint will be saved to the sdcard/dcim/camera/front directory.
- 2) The emergency recording file will be moved to the sdcard/dcim/camera/protect/keypoint directory.

Keypoint Protection Case3



Use a 10-second pre and post protection as an example: The keypoint occurs at 65s. The range of the emergency recording spans across two regular recording files.

A 20s emergency recording file will be created (10s before the event + 10s after the event). The file consists of 5 seconds from the previous file and the 15 seconds from the current file.

- 1) The normal recording file after the keypoint will be saved to the `sdcard/dcim/camera/front` directory.
- 2) The emergency recording file will be moved to the `sdcard/dcim/camera/protect/keypoint` directory.

1.4 Frequently Asked Questions/Troubleshooting

1.4.1 Common DVR Issue Analysis

1.4.1.1 DVR Crashes due to Unconfigured Metadata Resolution

Error Log: create stream fail

```
01-08 14:23:38.652375 572 2400 E mtkcam-AppStreamMgr: [0-ConfigHandler::checkStream]
unsupported size 2560x1920 for format 0x22/rotation:0 - {.v3_2 = {.id = 2, .streamType =
OUTPUT, .width = 2560, .height = 1920, .format = IMPLEMENTATION_DEFINED, .usage =
CPU_READ_NEVER | CPU_WRITE_NEVER | VIDEO_ENCODER (0x10000), .dataSpace = UNKNOWN |
STANDARD_UNSPECIFIED | TRANSFER_UNSPECIFIED | RANGE_UNSPECIFIED | BT709 (0x104), .rotation =
ROTATION_0}, .physicalCameraId = "", .bufferSize = 0}
(checkStream) {#619:vendor/mediatek/proprietary/hardware/mtkcam3/main/hal/device/3.x/app/AppS
treamMgr.ConfigHandler.cpp}
01-08 14:23:38.652400 572 2400 E mtkcam-AppStreamMgr: [0-ConfigHandler::checkStreams]
streams[id:2] has a bad status: -22(Invalid argument)
(checkStreams) {#652:vendor/mediatek/proprietary/hardware/mtkcam3/main/hal/device/3.x/app/App
StreamMgr.ConfigHandler.cpp}
```

Root Cause:

The camera HAL checks the metadata configuration based on the resolution and format passed down from the upper layer and then creates a stream. If it is not configured in the metadata, it will lead to the failure of stream creation.

Solution:

Add the configuration in `mt6771/hal/imgsensor_metadata/common/config_static_metadata_scaler.h` (by default, the preview is in `yuv420_888` format, and DVR is in `IMPLEMENTATION_DEFINED` format, the camera HAL will convert it to `yv12`. Please add all formats for that resolution, for example, for `yv12` refer to the following)

```
CONFIG_ENTRY_VALUE(HAL_PIXEL_FORMAT_YV12, MINT64)
CONFIG_ENTRY_VALUE( 2560 MINT64)
CONFIG_ENTRY_VALUE( 1920, MINT64)
CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT, MINT64)
CONFIG_ENTRY_VALUE(66666666, MINT64) // frame duration
CONFIG_ENTRY_VALUE(33333333, MINT64) // stall duration
```

1.4.1.2 Incorrect Format Configuration in Comprofile Leads to Recorded Videos Having Sound but No Image

Error Log:

```
CarCamDeviceClient: [0][startRecord] enter usage: 5, parmas: enable-file-time=1;preview-fps-range=25,25;recording-hint=true;drop-camera-frame=0;watermark-img-mode=off;watermark-text-mode=on;watermark-preview-en=1;watermark-timestamp-format=%Y-%m-%d %H:%M:%S;watermark-timestamp-ms=0;watermark-font-file=/system/fonts/NotoSansCJK-Regular.ttc;watermark-area=(250,739,540,803,1);watermark-text-size=41.33;watermark-text-x=0.0;watermark-text-y=41.33;watermark-area-ex1=(156,60,844,124,1);watermark-area-ex2=(156,136,844,200,1);preview-size=176x144;picture-size=176x144;watermark-text-color=-1;watermark-text-ex1=0.000000E 0.000000N;watermark-text-ex2=车牌: 车速:0km/h CH 1;free-size-limit=900;preview-frame-rate=25;record-id=Cam_0#Rec_0;video-param-camera-id=0;video-output-file=/storage/967A-A2AF/CH1;video-output-file-name=CH1_%Y%m%d%H%M%S%n;video-lock-file=/storage/967A-A2AF/protect;lock-file-name-prefix=Key;video-output-format=2;video-encoder=1;video-size=176x144;video-frame-rate=25;video-param-encoding-bitrate=2500000;audio-s
```

Root Cause:

The `camcorderProfile` is set with the video encoder format as `H263`, which causes the recorded video to lack video data.

Solution:

1. Change the resolution added by the customer in

`device\mediatek\corresponding_project\media_profiles.xml` to `h264` as follows:

```
<EncoderProfile quality="high" fileFormat="mp4" duration="30">
  <Video codec="h263"
    bitRate="17000000"
    width="1920"
    height="1080"
    frameRate="30" />
  <Audio codec="aac"
    bitRate="128000"
    sampleRate="48000"
    channels="1" />
</EncoderProfile>
```

2. Refer to the `camcorderMcam` demo where `camcorderProfile.videoCodec` is set to `MediaRecorder.VideoEncoder.H264`.

1.4.1.3 Audio Error Causes DVR to Crash

Error Log:

```
CarCamDeviceClient07-23 14:34:40.560212 632 5835 E AudioRecord: start(60): status -38
07-23 14:34:40.560237 632 5835 D AudioRecord: start(60): return status -38
07-23 14:34:40.560297 632 5835 D AudioRecord: ~AudioRecord(60): mStatus 0
```

Root Cause:

Audio start error leads to DVR start error.

Solution:

The audio owner indicates that there is only one audio path at the bottom layer, and attempting to start another path before the first has started will cause the start to fail. The APK should not handle each start record in a separate thread.

1.4.1.4 SD Card Errors Lead to Data Accumulation and DVR Errors

Error Log:

```
Line 4303: 06-10 18:36:53.491197 1044 22716 I ActivityManager: ntv ?? 700733:
smartplatformserver (pid 611) native
    Line 4695: 06-10 18:37:42.784050 1044 1141 I AnrManager: 48%
611/smartplatformserver: 24% user + 23% kernel / faults: 24655 minor 7193 major
Line 436043: 06-10 18:14:53.203727 611 2971 I CarCamDeviceClient:
[0][notifyStatusChanged] usage = 5, status=3, arg1=6, arg2=comment=sdcard_damaged , arg3:0
Line 499031: 06-10 18:15:43.163869 611 2969 I RecorderMgr: [Cam_1#Rec_0][recordMgrNotify]
param: comment=sdcard_damaged
Line 499034: 06-10 18:15:43.163988 611 2969 I CarCamDeviceClient:
[1][notifyStatusChanged] usage = 5, status=3, arg1=6, arg2=comment=sdcard_damaged , arg3:1
Line 505241: 06-10 18:15:48.164704 611 3061 E MPEG4WriterSmp:
[Cam_1#Rec_0][bitrateCheckThread] [1]preBitrate:500000, buffer so large,stop record,
mNotifySdcardDamaged:1
```

Root Cause:

SD card damage or other reasons cause the read/write speed to slow down, leading to excessive data accumulation and DVR error.

Solution:

Ask the customer to replace the SD card for testing, and when the mNotifySdcardDamaged message is triggered, refer to the VideoCallback registered in CarcoderDemoActivity.java and implement stop record for the sdcard_damaged message to prevent memory leak.

1.4.1.5 Resolution Exceeds Encoder Capability Causing DVR Errors

Error Log:

```
01-17 16:18:27.647951 567 3129 E MtkOmxVenc: [0xf01c2000] [ERROR] cannot support H.264
(2560x1920) encoder
01-17 16:18:27.647972 567 3129 E MtkOmxVenc: [0xf01c2000] [ERROR] cannot init encode
driver
```

Root Cause:

The maximum resolution supported by 8666/8667 VENC is 1080p.

Solution:

DVR recording needs to be set to 1080P or lower resolution.

1.4.1.6 Removing the SD Card during Recording

Error Log:

```
05-23 20:08:21.014271 2369 4131 W System.err: android.os.DeathObjectException //kk
05-23 20:08:21.014416 2369 4131 W System.err: at
android.os.BinderProxy.transactNative(Native Method)
05-23 20:08:21.014473 2369 4131 W System.err: at
android.os.BinderProxy.transact(BinderProxy.java:540)
05-23 20:08:21.014497 2369 4131 W System.err: at
com.mediatek.smartplatform.ICarCamDeviceUser$Stub$Proxy.stopRecord(ICarCamDeviceUser.java:78
4)
05-23 20:08:21.014521 2369 4131 W System.err: at
com.mediatek.smartplatform.SpmCameraDeviceImpl.stopRecord(SpmCameraDeviceImpl.java:766)
```

Root Cause:

When the SD card is removed, the APK receives a broadcast, ACTION_MEDIA_EJECT, and then the APK will perform the stop record action, and the lower layer stops recording. However, after vold issues this broadcast, it immediately kills the process still using the SD card, killing smartplatformserver, and then causing the stop recording to fail.

Solution:

In `/system/vold/utils.cpp`, at the very beginning of the function `killProcessesUsingPath`, add a 5-second wait:

```
if(sSleepOnUnmount) sleep(5);
```

Vold issues this broadcast and waits for the stop recording to finish, so vold will not kill smartplatformserver.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.